

ParcelFlow API

একটি কুরিয়ার প্রতিষ্ঠানের পুরো ডেলিভারি প্রক্রিয়া — বুকিং থেকে হাতে পৌঁছানো পর্যন্ত — JSON REST API হিসেবে। প্রতিটি স্ট্যাটাস পরিবর্তন একটি append-only history টেবিলে স্থায়ীভাবে লেখা হয়, যাতে কোন পার্সেল কখন কার হাতে ছিল তা কখনো হারিয়ে না যায়।

বিষয়	বিবরণ
Stack	Node 22 · Express 5 · PostgreSQL 16 · Prisma 6 · Zod 4
Endpoints	১৫টি — GET · POST · PATCH (কোনো DELETE নেই)
ডেটা মডেল	৩ টেবিল · ২ enum · ৭ index
ভূমিকা	৩ ধরনের — Customer · Delivery Staff · Admin
Status flow	৫ ধাপ · ৪টি বৈধ transition
Deployment	Render · Neon PostgreSQL · Upstash Redis

সূচিপত্র

(Word-এ খোলার পর পৃষ্ঠা নম্বর দেখতে এই তালিকায় ক্লিক করে F9 চাপুন)

প্রকল্প পরিচিতি

ParcelFlow একটি parcel delivery management REST API। কুরিয়ার প্রতিষ্ঠানের দৈনন্দিন কাজগুলো — পার্সেল বুক করা, ডেলিভারি স্টাফ assign করা, পার্সেলের অবস্থা ধাপে ধাপে এগিয়ে নেওয়া, এবং প্রতিটি পরিবর্তনের স্থায়ী রেকর্ড রাখা — এই পুরো প্রক্রিয়াটিকে একটি JSON API হিসেবে তৈরি করা হয়েছে।

তিন ধরনের ব্যবহারকারী একই ডেটা থেকে ভিন্ন ভিন্ন জিনিস চান। একজন **customer** পার্সেল বুক করেন এবং জানতে চান সেটি এখন কোথায়। একজন **delivery staff** পার্সেলটি বহন করেন এবং কী ঘটল তা রেকর্ড করেন। একজন **admin** পুরো কার্যক্রম পরিচালনা করেন, কাজ বন্টন করেন এবং পরিসংখ্যান দেখেন। API-এর মূল কাজ হলো একই ডেটার উপর এই তিনটি ভিন্ন দৃষ্টিভঙ্গি নিরাপদে সরবরাহ করা।

এটি সচেতনভাবে একটি backend-only প্রকল্প। এতে কোনো user interface নেই, email বা SMS পাঠানোর ব্যবস্থা নেই, payment handling নেই, এবং GPS tracking নেই। এটি একটি নির্দিষ্ট linear delivery pipeline-কে মডেল করে এবং সেই pipeline-এর প্রতিটি পদক্ষেপের নির্ভুল হিসাব রাখে।

যে নীতি পুরো প্রকল্পকে আকার দিয়েছে

পার্সেলের ইতিহাস অবশ্যই নির্ভরযোগ্য হতে হবে। এমন যেকোনো কিছু যা একটি ভুল, অনুপস্থিত বা duplicate history row তৈরি করতে পারে — সেটিকে edge case হিসেবে নয়, বরং **correctness bug** হিসেবে ধরা হয়েছে। এই নথির প্রায় প্রতিটি design decision এই একটি নীতি থেকেই এসেছে।

কী সমস্যার সমাধান করে

ছোট ও মাঝারি কুরিয়ার প্রতিষ্ঠানে পার্সেলের হিসাব সাধারণত ফোন কল, খাতা আর spreadsheet-এ রাখা হয়। এই পদ্ধতিতে চারটি সমস্যা বারবার ফিরে আসে:

- **দায়বদ্ধতা থাকে না।** একটি পার্সেল কখন কার হাতে ছিল, কে সেটির অবস্থা বদলেছে — এর কোনো স্থায়ী প্রমাণ থাকে না। বিরোধ হলে কারও কথার বিপরীতে কারও কথা।
- **যে কেউ যেকোনো কিছু বদলাতে পারে।** ভূমিকা অনুযায়ী কাজের সীমা না থাকায় একজন গ্রাহকও নিজের পার্সেলকে "delivered" দেখিয়ে দিতে পারেন।
- **ধাপ এড়িয়ে যাওয়া যায়।** পার্সেল pickup না হয়েই "delivered" হয়ে যেতে পারে, কারণ কোনো নিয়ম সেটিকে আটকায় না।
- **গ্রাহক অন্যের তথ্য দেখে ফেলেন।** একটি tracking নম্বর অনুমান করে অন্যের পার্সেলের তথ্য বের করে ফেলা যায়।

ParcelFlow এই চারটিকেই কাঠামোগতভাবে বন্ধ করে। প্রতিটি পরিবর্তন একই transaction-এ history-তে লেখা হয়; প্রতিটি endpoint ভূমিকা যাচাই করে; status পরিবর্তন একটি নির্দিষ্ট state machine মেনে চলে; এবং প্রতিটি database query-তে ownership শর্ত হিসেবে যুক্ত থাকে, ফলে নিজের নয় এমন কোনো row কখনো database থেকে বেরই হয় না।

০৩ / FEATURES

প্রধান ফিচারসমূহ

Authentication ও Authorization

- **Stateless JWT authentication** — HS256 algorithm এবং issuer ছোটোই pin করা, যা `alg: none` ও `algorithm-confusion` আক্রমণ বন্ধ করে।
- প্রতি **request-এ user পুনরায় পড়া হয়** — token-এর role claim বিশ্বাস করা হয় না, তাই কাউকে demote বা delete করলে সেটি পরের request থেকেই কার্যকর হয়।
- **দুই স্তরের authorization** — "এই role কি এই endpoint ডাকতে পারে?" middleware, উত্তর 403। "এই caller কোন record দেখতে পারে?" service-এর ভিতরে query scope, উত্তর 404।

- **Register করে admin হওয়া যায় না** — Zod `strictObject` অচেনা `role` key পেলে 422 দেয়, আর service-এ role সরাসরি `CUSTOMER` hard-code করা।

Parcel lifecycle

- **৫ ধাপের কঠোর linear flow** — ২৫টি সম্ভাব্য জোড়ার মধ্যে মাত্র ৪টি বৈধ। ধাপ এড়ানো, পিছিয়ে যাওয়া বা একই status আবার সেট করা — সবই 409 দিয়ে প্রত্যাখ্যাত।
- **Append-only audit trail** — প্রতিটি পরিবর্তন (কে, কখন, কোথা থেকে কোথায়) একই transaction-এ লেখা হয়। কোডের কোথাও এই টেবিলে update বা delete করা হয় না।
- **Optimistic locking** — দুজন staff একসাথে একই পার্সেল এগোলে conditional UPDATE-এ একজনের সাথে শূন্য row মেলে, transaction rollback হয়, তিনি 409 `STATUS_CONFLICT` পান।
- **অনুমান-অযোগ্য tracking code** — `crypto.randomInt` দিয়ে তৈরি, বিভ্রান্তিকর অক্ষর বাদ। Uniqueness database-এর UNIQUE constraint নিশ্চিত করে, collision হলে retry।
- **Filter • pagination • sort** — status, deliveryArea, pickupArea ও আংশিক trackingCode দিয়ে filter; page ও limit; চার রকম sort — সবই index-সমর্থিত।
- **Staff assign ও release** — Admin পার্সেল assign করেন; `null` পাঠালে assignment মুছে যায়। Staff-কে demote করলে তার অসম্পূর্ণ পার্সেলগুলো একই transaction-এ মুক্ত হয়।

Admin ও analytics

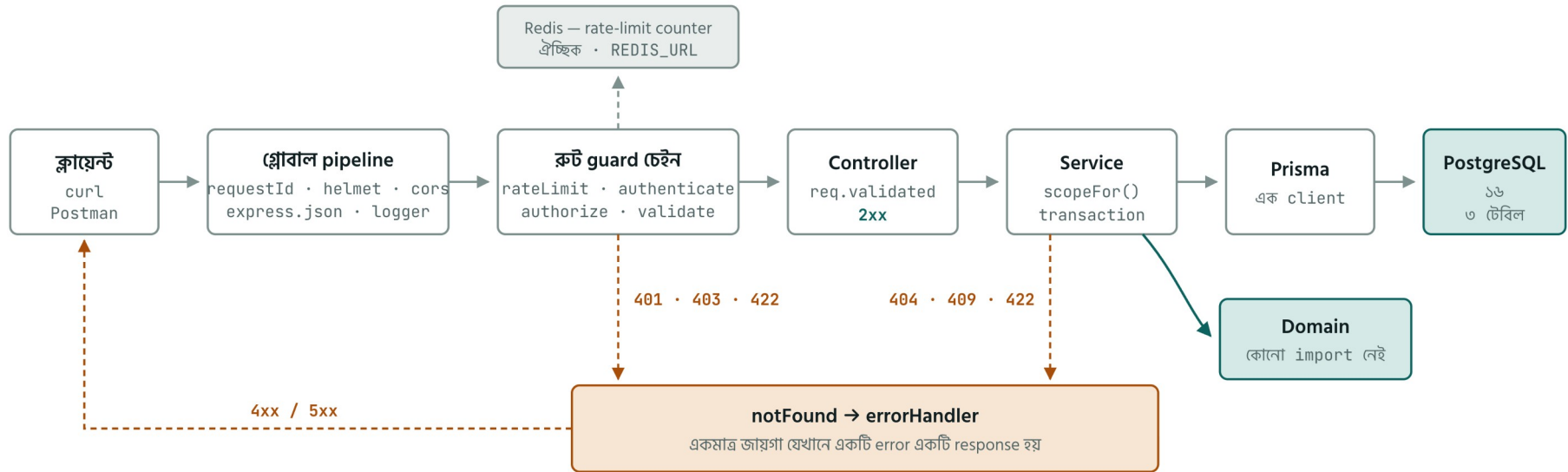
- **সম্পূর্ণ database-এ গণনা করা পরিসংখ্যান** — মোট পার্সেল, status অনুযায়ী ভাগ, unassigned সংখ্যা, delivery rate, গড় ডেলিভারি সময় ও শীর্ষ এলাকা। কোনো row memory-তে এনে গোনা হয় না।
- **ব্যবহারকারী ব্যবস্থাপনা** — role অনুযায়ী filter, নাম ও email-এ search, pagination। Role পরিবর্তন করা যায়, তবে নিজের role নয়।
- **সবসময় সম্পূর্ণ আকৃতি** — `groupBy` শূন্য-সংখ্যক key বাদ দেয়, তাই প্রতিটি status ও role শূন্য দিয়ে পূরণ করা হয়।

Operations

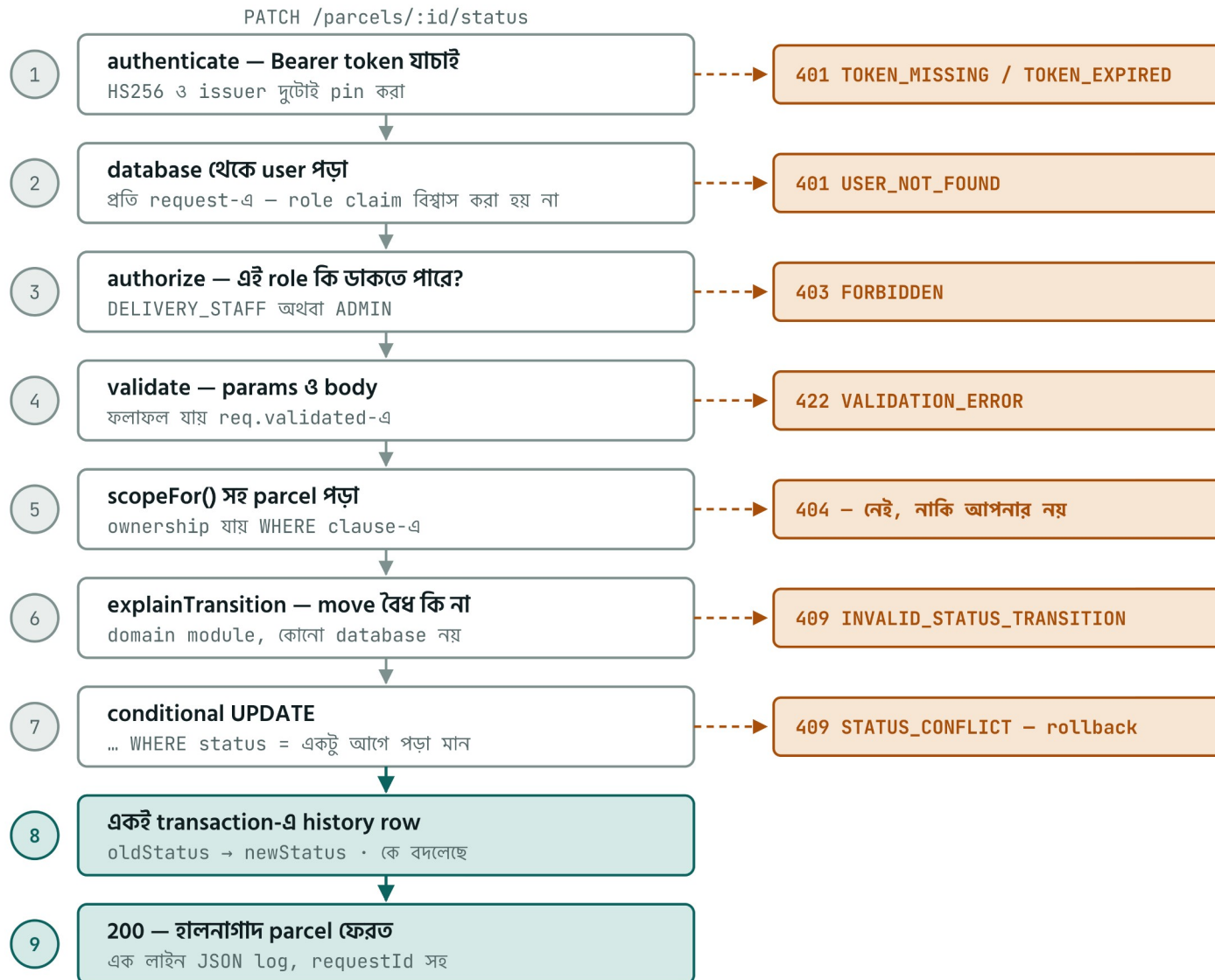
- **Boot-এ environment validation** — সব variable Zod দিয়ে যাচাই হয়। ভুল থাকলে কোন variable-এ সমস্যা তা ছাপিয়ে process বন্ধ হয়।
- **Request id ও structured log** — প্রতিটি request একটি correlation id পায়, যা `X-Request-Id` header ও প্রতিটি error body-তে থাকে। Log হয় এক লাইনের JSON।
- **দুই স্তরের health check** — `/health` database ছোঁয় না, তাই uptime monitor বিনা খরচে ডাকতে পারে। `/health/db` গভীর check করে এবং অচল হলে 503 দেয়।
- **Login ও tracking rate limiting** — Redis থাকলে counter সব instance-এ ভাগ হয়, না থাকলে in-memory। Login limiter validation-এর আগেই চলে।
- **Graceful shutdown** — SIGTERM পেলে নতুন connection বন্ধ, চলমান request শেষ, Prisma pool ছাড়া, তারপর exit — ১০ সেকেন্ডের কঠোর সময়সীমা সহ।
- **`npm run check`** — এক কমান্ডে পুরো stack যাচাই: Docker, দুটি container, port, process, database, একটি সত্যিকারের login এবং একটি authenticated read।

সিস্টেম আর্কিটেকচার

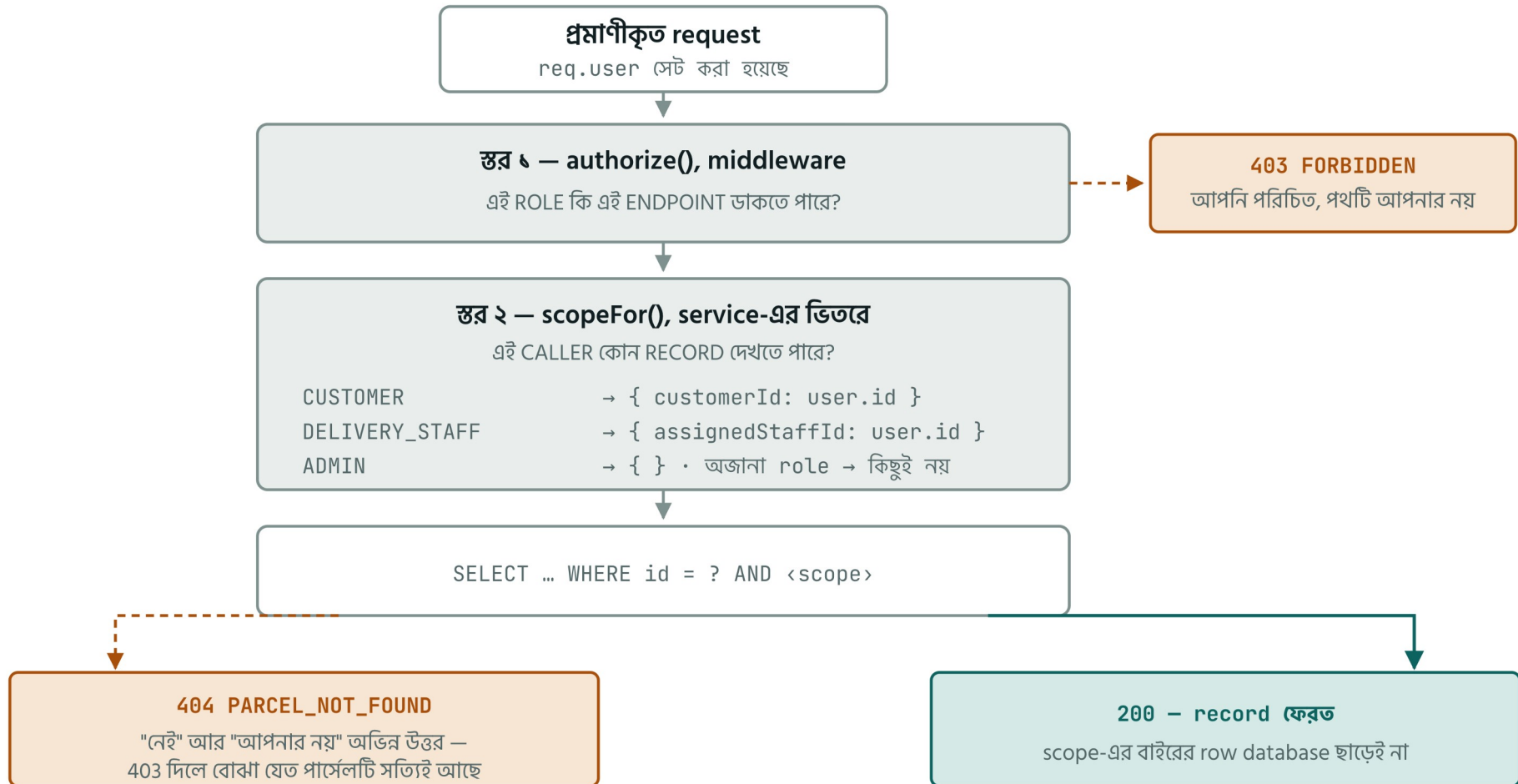
ছয়টি ডায়াগ্রাম, ছয়টি ভিন্ন প্রশ্নের উত্তর দেয়। প্রথমটি দেখায় কোডটি কীভাবে সাজানো, দ্বিতীয়টি দেখায় একটি request-এর ভিতরে আসলে কী ঘটে, আর বাকিগুলো ডেটা, নিয়ম, অনুমতি ও স্থাপনার ছবি।



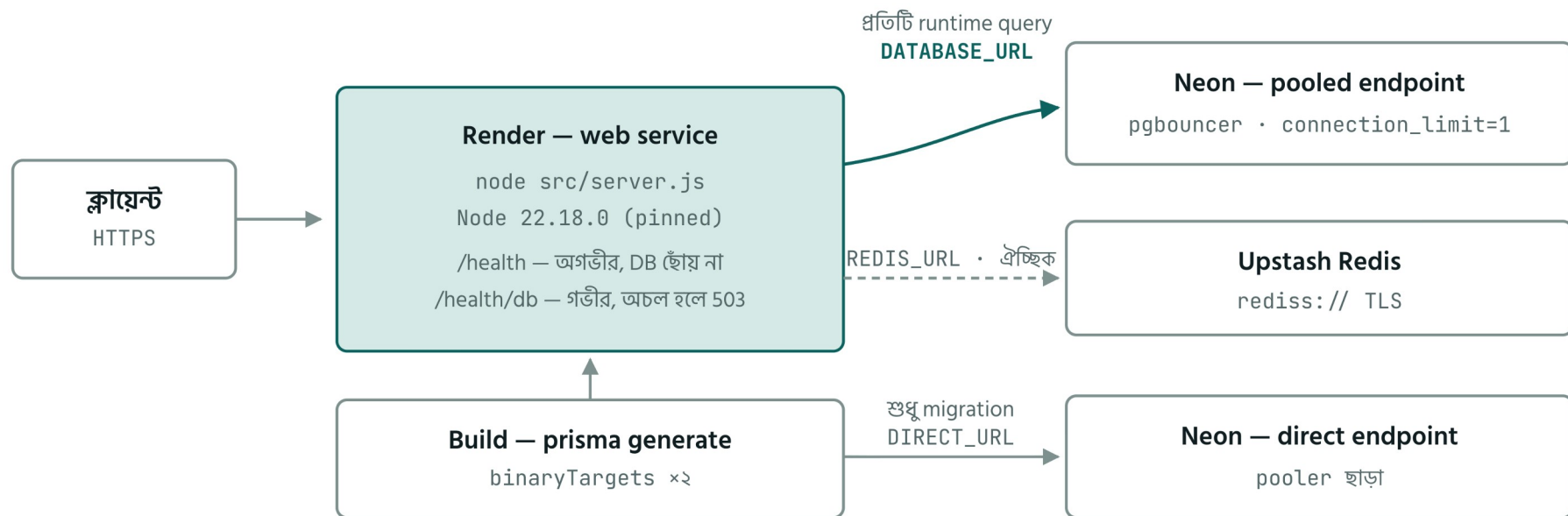
ডায়াগ্রাম ১ – সিস্টেম আর্কিটেকচার। অনুরোধ বাঁ থেকে ডানে চারটি স্তর পেরোয়: global pipeline, রুট-নির্দিষ্ট guard চেইন, controller, service। Service শুধু দুই দিকে তাকায় — নিয়মের জন্য domain module, আর ডেটার জন্য Prisma। যেকোনো স্তরে যা-ই ব্যর্থ হোক, সেটি একটিমাত্র জায়গায় গিয়ে response-এ পরিণত হয়।



ডায়াগ্রাম ২ – একটি request-এর সম্পূর্ণ যাত্রা। PATCH /parcels/:id/status কোডবেসের সবচেয়ে সমৃদ্ধ পথ — এটি authentication, দুই স্তরের authorization, domain নিয়ম, optimistic concurrency এবং audit trail — সবগুলোকেই ছোঁয়। বাঁয়ের কলাম সফল পথ, ডানের প্রতিটি বাক্স একটি ব্যর্থতার নির্দিষ্ট উত্তর।



ডায়াগ্রাম ৩ – অনুমতির দুই স্তর। "এই role কি এই endpoint ডাকতে পারে" এবং "এই caller কোন record দেখতে পারে" — দুটি আলাদা প্রশ্ন, দুই জায়গায়, দুই রকম উত্তর। Ownership সরাসরি WHERE clause-এ যায়, তাই কোথাও fetch-পরবর্তী ownership check নেই — ভুলে যাওয়ারও কিছু নেই।

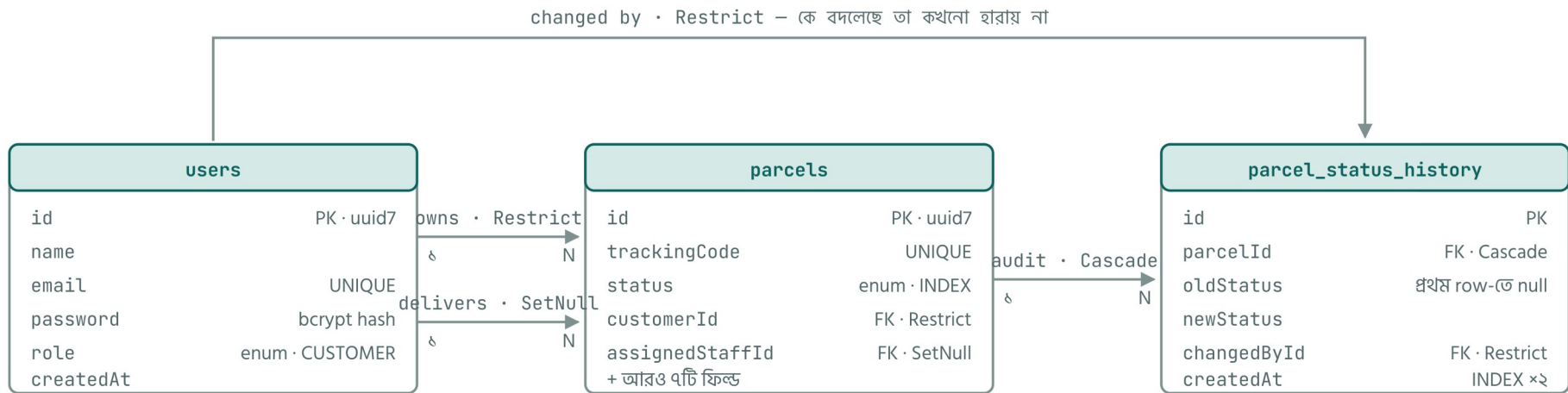


ডায়াগ্রাম ৪ – ডিপ্লয়মেন্ট। দুটি database URL ইচ্ছাকৃত: Neon-এর pooled endpoint pgBouncer চালায়, যেটি DDL চালাতে পারে না — তাই migration যায় সরাসরি endpoint-এ আর প্রতিটি runtime query যায় pooler-এর মধ্য দিয়ে। দুটি Prisma binary target দরকার, কারণ build container Debian আর ডেভেলপারের মেশিন নয়। Render-এর সামনে কোনো CDN, WAF বা proxy স্তর নেই।

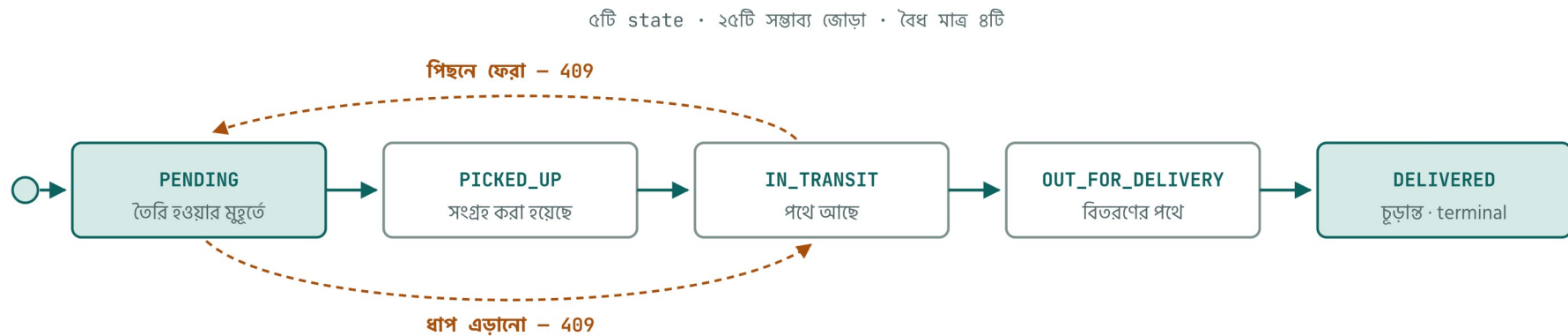
০৫ / DATA MODEL

ডেটা মডেল

তিনটি টেবিল, দুটি enum, সাতটি index। primary key হিসেবে UUID v7 ব্যবহার করা হয়েছে, v4 নয় — v7-এর উপরের bit-গুলোতে timestamp থাকে, ফলে তৈরি হওয়া id মোটামুটি সময়ের ক্রমে সাজানো থাকে এবং নতুন row index-এর শেষে বসে, পুরো index জুড়ে ছড়িয়ে পড়ে না।



ডায়াগ্রাম ৫ — ডেটা মডেল। লক্ষ্য করুন foreign key-এর তিনটি ভিন্ন আচরণ, এবং প্রতিটির কারণ আলাদা। মালিকের ক্ষেত্রে Restrict — পার্সেল কখনো মালিকহীন হতে পারে না। ডেলিভারি স্টাফের ক্ষেত্রে SetNull — কর্মী চলে গেলে পার্সেল মুক্ত হবে, আটকে যাবে না। History-র ক্ষেত্রে Cascade পার্সেলের দিকে, কিন্তু ব্যবহারকারীর দিকে Restrict — যে audit trail ভুলে যায় কে কাজটি করেছিল, সেটি আর audit trail নয়।



ডায়াগ্রাম ৬ – স্ট্যাটাস স্টেট মেশিন। পাঁচটি state থেকে ২৫টি ক্রমিক জোড়া সম্ভব; বৈধ মাত্র চারটি। একই status আবার সেট করাও প্রত্যাখ্যাত — নিরীহ no-op হিসেবে মেনে নিলে audit trail-এ একটি অর্থহীন row লেখা হতো, আর সেই trail-ই এখানে রক্ষা করার জিনিস। নিয়মগুলো একটি frozen object হিসেবে এমন একটি module-এ থাকে যা কিছুই import করে না।

প্রয়োজনীয় অন্যান্য তথ্য

Response format — একটিই খাম, প্রতিটি endpoint-এ

প্রতিটি response একই আকৃতির। ফলে client-এর একটিমাত্র parser দিয়ে সব উত্তর সামলানো যায়, endpoint-ভেদে আলাদা special case লাগে না। `meta` শুধু list endpoint-গুলোতে থাকে।

সফল response	ব্যর্থ response
<pre>{ "success": true, "data": { ... }, "meta": { "page": 1, "limit": 20, "total": 7, "totalPages": 1 } }</pre>	<pre>{ "success": false, "error": { "code": "INVALID_STATUS_TRANSITION", "message": "...", "details": { ... }, "requestId": "78ce46f6-..." } }</pre>

Client-দের জন্য নিয়ম: `code` দেখে সিদ্ধান্ত নিন, `message` দেখে নয়। বার্তার ভাষা বদলাতে পারে, কিন্তু `code` বদলায় না। প্রতিটি error-এ `requestId` থাকে এবং সেটি `X-Request-Id` header-এর সাথে মেলে, তাই কেউ কোনো সমস্যার কথা জানালে সেটি log-এ হুবহু খুঁজে পাওয়া যায়।

HTTP status ও প্রধান error code

Status	কখন	প্রধান code
200	পড়া বা হালনাগাদ সফল	—
201	নতুন কিছু তৈরি হয়েছে	—
401	Token নেই, ভাঙা, মেয়াদোত্তীর্ণ বা অবৈধ	TOKEN_MISSING · TOKEN_EXPIRED · INVALID_CREDENTIALS
403	পরিচিত caller, কিন্তু এই endpoint তার জন্য নয়	FORBIDDEN
404	নেই, অথবা caller-এর scope-এর বাইরে	PARCEL_NOT_FOUND · ROUTE_NOT_FOUND

Status	কখন	প্রধান code
409	দ্বন্দ্ব — ডুপ্লিকেট, অবৈধ transition, বা race	EMAIL_TAKEN · INVALID_STATUS_TRANSITION · STATUS_CONFLICT
413	Body ১০০kb-এর বেশি	PAYLOAD_TOO_LARGE
422	Validation ব্যর্থ, বা ভুল ধরনের staff	VALIDATION_ERROR · NOT_DELIVERY_STAFF
429	Rate limit অতিক্রম	TOO_MANY_LOGIN_ATTEMPTS
500	অপ্রত্যাশিত — বার্তা সবসময় সাধারণ	INTERNAL_ERROR
503	Database-এ পৌঁছানো যাচ্ছে না	DATABASE_UNAVAILABLE

ভূমিকা ও দৃশ্যমানতা

Role	কী দেখতে পায়	তৈরি	Status এগোনো	/admin
CUSTOMER	নিজের পার্সেল	হ্যাঁ	না	না
DELIVERY_STAFF	নিজের উপর assign করা পার্সেল	না	নিজেরগুলো	না
ADMIN	সবকিছু	হ্যাঁ	হ্যাঁ	হ্যাঁ

সবকটি endpoint

Method	Path	কে ডাকতে পারে	মন্তব্য
GET	/	সবাই	সার্ভিসের নাম, সংস্করণ, route prefix
GET	/health	সবাই	অগভীর — database ছোঁয় না
GET	/health/db	সবাই	গভীর — অচল হলে 503
POST	/auth/register	সবাই	সবসময় CUSTOMER তৈরি হয়
POST	/auth/login	সবাই	Rate limited
GET	/auth/me	যেকোনো লগইনকৃত	Token যার, তার তথ্য

Method	Path	কে ডাকতে পারে	মন্তব্য
POST	/parcels	Customer · Admin	PENDING-এ তৈরি, প্রথম history row সহ
GET	/parcels	যেকোনো role	Filter · pagination · sort, scope অনুযায়ী
GET	/parcels/:id/history	যেকোনো role	পুরনো থেকে নতুন ক্রমে
GET	/parcels/:trackingCode	যেকোনো role	Rate limited · history সহ
PATCH	/parcels/:id/status	Staff · Admin	State machine ও optimistic lock
PATCH	/parcels/:id/assign	Admin	null পাঠালে assignment মুছে যায়
GET	/admin/stats	Admin	৭টি query, সবই database-এ গণনা
GET	/admin/users	Admin	Filter · search · pagination
PATCH	/admin/users/:id/role	Admin	নিজের role বদলানো যায় না

পনেরোটি endpoint, সবগুলোই GET, POST বা PATCH — কোনো DELETE নেই। এটি ইচ্ছাকৃত: যে সিস্টেমের মূল প্রতিশ্রুতি একটি অপরিবর্তনীয় ইতিহাস, সেখানে মুছে ফেলার পথ যোগ করার আগে archive বনাম delete-এর প্রশ্নটি আলাদাভাবে ভাবা দরকার।

চালানোর ধাপ

১. প্রস্তুতি — `npm install`, তারপর `cp .env.example .env`। Node ২২ বা তার নতুন সংস্করণ লাগবেই, কারণ অ্যাপটি `process.loadEnvFile()` ব্যবহার করে।
২. Secret তৈরি — অন্তত ৩২ অক্ষরের একটি random string তৈরি করে `JWT_SECRET`-এ বসান। ছোট হলে সার্ভার চালুই হবে না।
৩. Database — `npm run db:up` দিয়ে Postgres ও Redis container চালু করুন, তারপর `npm run db:migrate`।

8. **Seed** — `npm run db:seed` — পাঁচজন ব্যবহারকারী এবং সাতটি পার্সেল তৈরি হয়, প্রতিটির সম্পূর্ণ history সহ। আবার চালালেও নিরাপদ।

৫. **চালু** — `npm run dev`, এবং `npm run check` দিয়ে পুরো stack এক নজরে যাচাই করুন।

Seed করা অ্যাকাউন্ট: একজন ADMIN (`admin@parcelflow.dev`), দুজন DELIVERY_STAFF

(`rahim.staff@`, `karim.staff@`) এবং দুজন CUSTOMER (`ayesha.customer@`, `tanvir.customer@`)।

পাসওয়ার্ড `.env`-এর `SEED_*_PASSWORD` variable থেকে আসে — এগুলো শুধুই development-এর সুবিধার

জন্য, প্রকাশ্য কোনো পরিবেশে seed করার আগে অবশ্যই বদলাতে হবে।

যেসব সমস্যার সম্মুখীন হয়েছি, ও সমাধান

নিচের প্রতিটি ভুক্তির পাশে কোড ফাইল বা commit-এর উল্লেখ আছে। অর্থাৎ প্রতিটি দাবি repository-তে যাচাই করা যায় — কোনোটিই স্মৃতি থেকে লেখা নয়।

১. দুজন staff একসাথে একই পার্সেল এগোলে ইতিহাসে দুটি row লেখা হতো

দুটি request একই সময়ে `status` পড়ে, দুটোই validation পাস করে, দুটোই লিখে ফেলে — ফলে audit trail মিথ্যা হয়ে যায়। **সমাধান:** UPDATE-টিকে শর্তসাপেক্ষ করা হয়েছে — `WHERE id = ? AND status =` `<একটু আগে পড়া মান>`। কেউ আগে বদলে ফেললে শূন্য row মেলে, transaction rollback হয়, এবং দ্বিতীয়জন 409 `STATUS_CONFLICT` পান। কোনো lock ধরে রাখতে হয় না।

```
parcels.service.js - updateStatus()
```

২. Tracking code অনুমান করে অন্যের পার্সেল বের করা যেত

`Math.random()` অনুমানযোগ্য অবস্থা থেকে seed হয়, তাই তা দিয়ে তৈরি code ধারাবাহিকভাবে অনুমান করা সম্ভব। **সমাধান:** `crypto.randomInt`, এবং বর্ণমালা থেকে বিভ্রান্তিকর অক্ষর (I, L, O, 0, 1) বাদ — যাতে লেবেল থেকে হাতে টাইপ করলেও ভুল না হয়। আলাদা সমস্যা ছিল uniqueness: আগে খুঁজে দেখে তারপর insert করা একটি race। তাই সরাসরি insert করে UNIQUE constraint-কে সংঘর্ষ ধরতে দেওয়া হয়, আর সংঘর্ষ হলে পাঁচবার পর্যন্ত retry।

```
utils/trackingCode.js · parcels.service.js - createParcel()
```

৩. Login-এর উত্তর আসতে কত সময় লাগে, তা দেখেই বোঝা যেত email নিবন্ধিত কি না

অচেনা email-এ কোনো bcrypt তুলনা হয় না, তাই উত্তর আসে মাইক্রোসেকেন্ডে; সঠিক email-এ তুল পাঁচওয়ার্ড দিলে bcrypt-এর জন্য ~১০০ms লাগে। শুধু এই সময়ের পার্থক্য দিয়েই কোন ঠিকানাগুলো নিবন্ধিত তা বের করে ফেলা যায়। **সমাধান:** অচেনা email-এর পথেও একটি dummy hash-এর বিপরীতে bcrypt তুলনা চালানো হয়, এবং দুই ক্ষেত্রেই হুবহু একই 401 `INVALID_CREDENTIALS` ফেরে।

```
auth.service.js - DUMMY_HASH, login()
```

৪. Register করার সময় role পাঠিয়ে admin হয়ে যাওয়া যেত

`/auth/register` unauthenticated। এটি যদি body থেকে role পড়ে, তবে এটি কার্যত একটি admin-অ্যাকাউন্ট কারখানা। **সমাধান:** দুটি স্তর, যার কোনোটির উপর একা নির্ভর করা হয়নি — `Zod schema strictObject`, তাই অচেনা `role` key দেয় (নীচের বাদ যায় না); আর `service`-এ role সরাসরি `CUSTOMER` লেখা।

```
auth.schemas.js – registerSchema · auth.service.js – register()
```

৫. 403 না 404 — উত্তরটাই ফাঁস করে দিচ্ছিল পার্সেলটি আদৌ আছে কি না

অন্যের পার্সেলে 403 দিলে সেটিই নিশ্চিত করে যে ওই tracking code-এ একটি পার্সেল আছে। **সমাধান:** `ownership`-কে permission check নয়, বরং query শর্ত হিসেবে ধরা হয়েছে — `scopeFor()` প্রতিটি `read`-এর `WHERE` clause-এ যুক্ত হয়। ফলে `scope`-এর বাইরের row database ছাড়েই না, এবং "নেই" ও "আপনার নয়" — দুটোই অভিন্ন 404।

```
parcels.service.js – scopeFor()
```

৬. Proxy-র পিছনে আসল client IP কোনটি, তা বের করতে পারছিলাম না

Render-এ deploy করার পর rate limiter ভুল IP-তে গণনা করছিল, ফলে একই সীমা কখনো সবার উপর, কখনো কারও উপরই প্রয়োগ হচ্ছিল না। **যেভাবে এগিয়েছি:** প্রথমে Redis যোগ করা হলো যাতে counter সব instance-এ ভাগ হয়; তারপর `passOnStoreError`, যাতে Redis-এর সাময়িক সমস্যায় পুরো API 500 না দেয়; তারপর একটি অস্থায়ী debug endpoint বসিয়ে দেখা হলো Render আসলে কোন IP পাঠাচ্ছে; সবশেষে সেটি সরিয়ে `trust proxy` ঠিক করা হলো। ছয় মিনিটে তিনটি commit — production-এ proxy debug করা আসলে এভাবেই দেখায়।

```
commits eaaa8f1 → 98bd6dc → eaf6d7c → 1f90d4e (২৮ আগস্ট)
```

৭. Neon-এ migration চলছিল না

Neon-এর pooled endpoint `pgBouncer` চালায় `transaction mode`-এ, আর সেই mode-এ DDL চালানো যায় না — `prisma migrate` ব্যর্থ হচ্ছিল। **সমাধান:** দুটি আলাদা connection string। `DATABASE_URL` (pooled) প্রতিটি runtime query-র জন্য, আর `DIRECT_URL` (direct) শুধু migration-এর জন্য।

```
prisma/schema.prisma – datasource db
```

৮. Deploy করার পর Prisma engine চলছিল না

Build container Debian-ভিত্তিক, আর ডেভেলপমেন্ট মেশিন নয় — build-এর সময় তৈরি query engine runtime image-এর সাথে মিলছিল না। **সমাধান:** `binaryTargets = ["native", "debian-openssl-3.0.x"]` — দুটি target-এর জন্যই engine তৈরি হয়।

```
prisma/schema.prisma — generator client
```

৯. Neon-এ migrate করার পর seed করা login কাজ করছিল না

Migration হোস্টেড database-এ schema তৈরি করেছিল ঠিকই, কিন্তু টেবিলগুলো ছিল খালি — seed তখনো শুধু local database-এ চলেছিল। ফলে login করলে 401 আসছিল, যা দেখে মনে হচ্ছিল auth-এ সমস্যা, অথচ সমস্যাটি ছিল ডেটার। **সমাধান:** seed script-কে হোস্টেড instance-এর দিকে চালানো। Script-টি বারবার চালানো নিরাপদ — ব্যবহারকারী email দিয়ে upsert হয়, আর পার্সেল কেবল টেবিল খালি থাকলেই তৈরি হয়।

```
prisma/seed.js
```

১০. একই পাসওয়ার্ডের দুটি ভিন্ন রূপে একই অ্যাকাউন্ট খুলে যেতে পারত

bcrypt কেবল প্রথম ৭২ বাইট hash করে, বাকিটা চুপচাপ উপেক্ষা করে। অর্থাৎ ৮০ অক্ষরের দুটি ভিন্ন পাসওয়ার্ড, যাদের প্রথম ৭২ বাইট এক, একই hash তৈরি করবে। **সমাধান:** schema-তেই `.max(72)`, যাতে সীমাটি স্পষ্ট এবং যাচাইকৃত হয়, লুকানো আচরণ না থাকে।

```
auth.schemas.js — passwordSchema
```

১১. Prisma দুটি User সম্পর্ককে আলাদা করতে পারছিল না

একটি পার্সেল দুইভাবে ব্যবহারকারীর সাথে যুক্ত — মালিক হিসেবে এবং যাকে assign করা হয়েছে তার হিসেবে। দুটি relation একই দুই model-এর মধ্যে হওয়ায় Prisma schema validate করতে পারছিল না।

সমাধান: নামযুক্ত relation — `ParcelCustomer` ও `ParcelAssignedStaff` — দুই প্রান্তেই একই নাম দিয়ে।

```
prisma/schema.prisma — model User, model Parcel
```

১২. Express 5-এ req.query-তে লেখা যায় না

Express 5-এ `req.query` একটি read-only getter হয়ে গেছে, তাই পুরনো অভ্যাস — validate করে parsed মান দিয়ে `req.query` overwrite করা — আর কাজ করে না। **সমাধান:** parsed ফলাফল `req.validated-`

তে লেখা হয়। ফলে raw request অক্ষত থাকে, আর validation-পরবর্তী প্রতিটি মান আগে থেকেই coerce ও trim করা — এটি বাধ্যতা থেকে শুরু হলেও শেষ পর্যন্ত ভালো নকশা হয়েছে।

```
middleware/validate.js
```

১৩. Free-tier build machine-এ native bcrypt build হচ্ছিল না

Native `bcrypt`-এর install-এর সময় C++ toolchain দরকার, যা free-tier build container-এ প্রায়ই থাকে না।

সমাধান: `bcryptjs` — সম্পূর্ণ JavaScript-এ লেখা, কোনো native build ধাপ নেই। খরচ বাস্তব: এটি native binding-এর চেয়ে ধীর। Cost factor configuration থেকে আসে, তাই test কমাতে পারে এবং production বাড়াতে পারে, কোড না বদলেই।

```
utils/password.js
```

১৪. পরিসংখ্যানে শূন্য-সংখ্যক status উধাও হয়ে যাচ্ছিল

Prisma-র `groupBy` শুধু সেই মানগুলোর row ফেরত দেয় যেগুলো অন্তত একবার আছে। ফলে কোনো পার্সেল `IN_TRANSIT` না থাকলে সেই key উত্তরেই থাকত না, এবং প্রতিটি client-কে অনুপস্থিত key সামলাতে হতো। **সমাধান:** `toCountMap()` প্রতিটি সম্ভাব্য key শূন্য দিয়ে পূরণ করে, তাই উত্তরের আকৃতি সবসময় একই।

```
admin.service.js — toCountMap()
```

১৫. Staff-কে demote করলে তার পার্সেলগুলো আটকে যেত

Role বদলে গেলেও পার্সেল তার নামেই assign থাকত, অথচ সে আর সেগুলোর status বদলাতে পারত না — পার্সেলগুলো কার্যত অদৃশ্য হয়ে যেত। **সমাধান:** role পরিবর্তন এবং তার অসম্পূর্ণ পার্সেল মুক্ত করা — দুটোই একই transaction-এ। উত্তরে কতগুলো পার্সেল মুক্ত হলো তাও ফেরত আসে।

```
admin.service.js — updateUserRole()
```

১৬. শেষ admin নিজেকে demote করে সবাইকে লক করে ফেলতে পারত

Role পরিবর্তনের endpoint-এ নিজের id দিলে সিস্টেমে আর কোনো admin না-ও থাকতে পারত, এবং কেউ আর কাউকে promote করতে পারত না। **সমাধান:** caller-এর নিজের id-তে এই কাজ সরাসরি নিষিদ্ধ — 409

```
CANNOT_CHANGE_OWN_ROLE
```

```
admin.service.js — updateUserRole()
```

১৭. Uptime monitor প্রতি ৩০ সেকেন্ডে একটি database connection খরচ করছিল

একটিমাত্র health endpoint যদি database ছোঁয়, তবে monitoring নিজেই একটি ধারাবাহিক লোড — বিশেষ করে `connection_limit=1` pool-এ। আবার database না ছুঁলে অচল database ধরাই পড়ে না। **সমাধান:** দুটি আলাদা প্রশ্ন, দুটি আলাদা endpoint — `/health` শুধু বলে process বেঁচে আছে, `/health/db` সত্যিই query করে এবং অচল হলে 503 দেয়।

`src/app.js`

যেগুলো শুধু আপনিই জানেন

উপরের প্রতিটি ভুক্তি কোড বা commit থেকে যাচাই করা। কিন্তু কিছু অভিজ্ঞতা repository-তে কোনো চিহ্ন রাখে না — যা শিখতে সময় লেগেছে, যা একবার লিখে মুছে আবার লিখেছেন, বা যেটি বুঝতে ডকুমেন্টেশন ঘাঁটতে হয়েছে। viva-তে এই উত্তরগুলোই সবচেয়ে বিশ্বাসযোগ্য শোনায়, তাই নিচের জায়গাগুলো নিজের ভাষায় পূরণ করুন।

পূরণ করুন	আপনার উত্তর
১ – যা শিখতে সবচেয়ে বেশি সময় লেগেছে যেমন: Prisma-র transaction API, Zod-এর pipe/transform, অথবা Express 5-এর নতুন আচরণ।	
২ – যে সিদ্ধান্ত একবার নিয়ে পরে বদলেছেন যেমন: প্রথমে অন্য কোনো কাঠামো বা লাইব্রেরি বেছে নিয়ে পরে সরে আসা।	
৩ – সবচেয়ে বিরক্তিকর bug যেটি ধরতে সবচেয়ে বেশি সময় গেছে, এবং শেষ পর্যন্ত কারণটি কী ছিল।	

ব্যবহৃত প্রযুক্তি

প্রতিটি নির্বাচনের পিছনে একটি নির্দিষ্ট কারণ আছে, এবং কয়েকটি ক্ষেত্রে সেই কারণটি হলো একটি **dependency** এড়ানো। প্রকল্পে **dotenv** নেই, **nodemon** নেই, এবং **async route wrapper**-ও নেই — কারণ Node ২২ ও Express ৫ এই তিনটি কাজই নিজেরাই করে।

Runtime ও framework

প্রযুক্তি	সংস্করণ	কেন এটি
Node.js	22.18.0	<code>process.loadEnvFile()</code> ও <code>node --watch</code> built-in, তাই dotenv ও nodemon কোনোটিই লাগে না। সংস্করণটি <code>.node-version</code> -এ pin করা, কারণ এটি পছন্দ নয় — শর্ত।
Express	5.x	Express ৫ rejected promise স্বয়ংক্রিয়ভাবে error handler-এ পাঠায়, তাই একটিও route handler try/catch বা asyncHandler দিয়ে মোড়ানো নেই।

ডেটা

প্রযুক্তি	সংস্করণ	কেন এটি
PostgreSQL	16	Enum, সত্যিকারের transaction, composite index এবং database-স্তরের aggregate — চারটিই এখানে ব্যবহৃত হয়েছে। Audit trail-এর নির্ভরযোগ্যতা transaction-এর উপর দাঁড়িয়ে।
Prisma	6.19	Migration-এর ইতিহাস ফাইল হিসেবে থাকে; প্রতিটি query parameterised; <code>\$transaction</code> -এ interactive transaction লেখা যায়। Schema ফাইলটি নিজেই ডেটা মডেলের পাঠযোগ্য দলিল।
Neon	—	Serverless Postgres — pooled ও direct দুটি endpoint দেয়, যা

প্রযুক্তি	সংস্করণ	কেন এটি
		migration আর runtime query আলাদা রাখতে দেয়।

নিরাপত্তা ও যাচাই

প্রযুক্তি	সংস্করণ	কেন এটি
Zod	4.x	দুই জায়গায় — HTTP boundary-তে input যাচাই, আর boot-এ environment যাচাই। <code>strictObject</code> অচেনা key নীরবে ফেলে না দিয়ে প্রত্যাখ্যান করে, যা privilege escalation বন্ধ করার একটি স্তর।
jsonwebtoken	9.x	Algorithm ও issuer দুটোই pin করা যায় — এটাই <code>alg: none</code> ও algorithm-confusion আক্রমণ বন্ধ করার উপায়।
bcryptjs	3.x	সম্পূর্ণ JavaScript, তাই free-tier build container-এ কোনো native build ধাপ ব্যর্থ হওয়ার সুযোগ নেই। Cost factor configuration থেকে আসে।
helmet	8.x	নিরাপত্তা-সংক্রান্ত HTTP header-গুলো এক লাইনে। সাথে <code>x-powered-by</code> বন্ধ করা হয়েছে।
express-rate-limit	8.x	Login ও tracking endpoint-এ throttling। <code>rate-limit-redis</code> ও <code>ioredis</code> সাথে থাকলে counter সব instance-এ ভাগ হয়।

পরিবেশ ও টুলিং

প্রযুক্তি	কেন এটি
Docker Compose	Local Postgres ও Redis, দুটোই healthcheck সহ। একটি init script আলাদা test database তৈরি করে, যাতে test চালালে development-এর ডেটা মুছে না যায়।
Upstash Redis	Managed Redis TLS সহ। ঐচ্ছিক — না থাকলে rate limiter in-memory store-এ চলে, যা একটিমাত্র instance-এর জন্য সঠিক।
Render	Node অ্যাপের জন্য সরাসরি deploy। Platform নিজেই <code>PORT</code> ও অন্যান্য variable

প্রযুক্তি	কেন এটি
	inject করে, তাই production-এ .env পড়াই হয় না।
Postman	২৩টি সংরক্ষিত request — শুধু সফল পথ নয়, যেগুলো ব্যর্থ হওয়ার কথা সেগুলোও (403, 409, 422), যাতে নিয়মগুলো হাতে-কলমে দেখানো যায়।

সীমাবদ্ধতা ও পরবর্তী পরিকল্পনা

যে কাজগুলো সচেতনভাবে এই ধাপের বাইরে রাখা হয়েছে, গুরুত্ব অনুযায়ী সাজানো। প্রতিটির পাশে কী করতে হবে তা নির্দিষ্ট করে লেখা, যাতে এগুলো অস্পষ্ট ইচ্ছা নয় — কাজের তালিকা হয়ে থাকে।

১. Rate limiting আরও শক্ত করা

বর্তমানে limiter-এর bucket key প্রথমে `cf-connecting-ip` header থেকে নেওয়া হয়, আর header পাঠায় client নিজেই। এছাড়া `trust proxy` ৩-তে সেট, যেখানে এই deployment-এ hop আসলে একটি; এবং store-এ ত্রুটি হলে request পাস করে দেওয়া হয়। তিনটি মিলিয়ে limiter এখন কঠোর নিয়ন্ত্রণ নয়, বরং সাধারণ অপব্যবহারের বিরুদ্ধে একটি speed bump। **করণীয়:** platform-নির্ধারিত client IP-তে key করা, hop সংখ্যা বাস্তব মানে নামানো, এবং store ব্যর্থ হলে in-memory-তে ফিরে যাওয়া।

২. Registration হার্ডেন করা

`/auth/register`-এ কোনো rate limit নেই, এবং নিবন্ধিত ঠিকানার জন্য আলাদা 409 `EMAIL_TAKEN` ফেরে — অর্থাৎ login যে enumeration সাবধানে বন্ধ করেছে, register সেটি খুলে রাখে। পাসওয়ার্ডের কোনো ন্যূনতম দৈর্ঘ্যও এখন প্রয়োগ হয় না। **করণীয়:** limiter যোগ, ন্যূনতম দৈর্ঘ্য নির্ধারণ, এবং নিবন্ধিত ঠিকানার ক্ষেত্রেও একই আকৃতির উত্তর।

৩. Automated test লেখা

পুরো harness ইতিমধ্যেই তৈরি এবং অব্যবহৃত: `createApp()` আলাদা করা হয়েছে যাতে test port ছাড়াই অ্যাপ চালাতে পারে, `supertest` ইনস্টল করা, `TEST_DATABASE_URL` config-এ যুক্ত, Docker init script আলাদা test database বানায়, এবং rate limiter test পরিবেশে নিজেকে skip করে। যা নেই তা হলো test ফাইল আর একটি `npm test` script। **অগ্রাধিকার:** প্রথমে `parcelStatus.js`-এর transition টেবিল, তারপর প্রতিটি role-এর `scopeFor()` দৃশ্যমানতা, তারপর `updateStatus`-এর 409 race, তারপর `register/login`-এর enumeration আচরণ।

৪. assignStaff-এর দুটি race বন্ধ করা

এখানে পার্সেলের status ও লক্ষ্য ব্যবহারকারীর role পড়া হয়, তারপর লেখা হয় — তিনটি আলাদা statement-এ, কোনো transaction ছাড়া। পড়া আর লেখার মাঝে কর্মীকে demote করা হলে পার্সেল non-staff ব্যবহারকারীর নামে থেকে যেতে পারে, অথবা পার্সেল delivered হয়ে গেলে "delivered পার্সেল পুনরায় assign করা যায় না" নিয়মটি এড়িয়ে যায়। **করণীয়:** `updateStatus`-এ ব্যবহৃত একই কৌশল — UPDATE-কে পঠিত মানের উপর শর্তসাপেক্ষ করা।

৫. Error code-এর অসঙ্গতি দূর করা

`USER_NOT_FOUND` দুটি ভিন্ন status-এ ফেরে — `authenticate` থেকে 401, আর admin service থেকে 404। যেহেতু client-দের বলা হয়েছে message নয়, code দেখে সিদ্ধান্ত নিতে, একই code-এর দুই অর্থ সত্যিকারের অস্পষ্টতা। **করণীয়:** হয় আলাদা code, নয়তো একই status।

৬. ছোট ধারালো কোণাগুলো

`GET /parcels/<uuid>` tracking-code route-এ গিয়ে পড়ে এবং 404-এর বদলে 422 দেয়, কারণ id ও tracking code একই path অবস্থান ভাগ করে। `trackingLimiter` `authenticate`-এর আগে চলে, তাই একটি unauthenticated request-ও কারও কোটা খরচ করে। Tracking code তৈরির পাঁচটি চেষ্টা শেষ হলে raw P2002 বেরিয়ে 409 `DUPLICATE` হয়ে যায়, যা client-এর না-পাঠানো একটি মানের জন্য তাকেই দোষ দেয়। আর `cors()` এখন সব origin-এর জন্য খোলা।

৭. যেসব সুবিধা এখনো তৈরি হয়নি

বড় ডেটাসেটের জন্য cursor-ভিত্তিক pagination (offset গভীর page-এ ধীর হয়ে যায়); refresh token ও logout; `DELETE` বা archive endpoint — এখন একটিও নেই; এবং CI, অ্যাপের নিজস্ব `Dockerfile` ও repository-তে রাখা deploy configuration।

— নথির শেষ —

কারিগরি বিস্তারিত ইংরেজিতে docs/architecture.md-এ, যেখানে ২০টি Architecture Decision Record ও প্রতিটি ফিল্ড ও index-এর ব্যাখ্যা আছে। Endpoint-এর সম্পূর্ণ reference README.md-এ।